Digital Image Processing

Prof. Zohreh Azimifar

Homework 3

Topics: Hough Transform

Submission Format: PDF Report + Source Code (.zip)

May 2026

General Rules & Constraints

- **The "From Scratch" Rule:** All core algorithms must be implemented from scratch using fundamental mathematical and matrix operations.
    - **FORBIDDEN:** Built-in computer vision functions for filtering, edge detection, or Hough Transforms (e.g., cv2.Canny, cv2.Sobel, cv2.HoughLines, cv2.HoughCircles, scipy.signal.convolve2d). Using these for the core pipeline will result in an automatic **score of ZERO** for that section.
    - **ALLOWED:** cv2.imread, cv2.imwrite, cv2.cvtColor (for basic color space conversions only), numpy array operations, and matplotlib/imageio for plotting and GIF generation.
- **Vectorization & Performance:** Python nested for loops over image pixels are extremely slow. You are strictly required to use **NumPy vectorization** (e.g., array slicing, boolean masking, matrix multiplication) to optimize your execution time. Solutions relying entirely on slow, unoptimized loops will be penalized.
- **Execution Profiling:** You must calculate and print the execution time for each major part of your algorithm (e.g., Preprocessing, Edge Detection, Hough Voting).
- **Dataset:** All required image datasets are located in the images/ directory. You are required to process all images provided in their respective folders.

Part 1: Traffic Light Detection & State Classification (Hough Circles)

Input Directory: images/traffic_lights/

In this section, you will implement a highly optimized Hough Circle Transform to detect traffic lights and classify their current state.

Objectives:

1. **Custom Edge Detection:** Implement an edge detection algorithm from scratch. You may choose the method (e.g., Sobel, Prewitt, or full Canny with Non-Maximum Suppression and Hysteresis, as taught in class).
2. **Optimized 3D Hough Circle Transform:** Implement the 3D Hough space (a, b, r) voting scheme from scratch for circles of unknown radius.
3. **State Classification:** Analyze the color inside the detected circles to classify the state of the traffic light. You must detect active lights (**RED, YELLOW, GREEN**) as well as inactive/turned-off lamps (**UNLIT**).

Requirements & Hints:

- **Gradient Optimization (Required):** As taught in the lectures (*"Using Gradient Information"*), a brute-force 3D Hough Circle implementation $O(E \times R \times 2\pi r)$ is too slow. You **must** utilize the gradient direction (θ) calculated during your edge detection to reduce the free parameters and cast votes only along the normal line of the edge.
- **Output:** Draw the detected circles on the original image and overlay text indicating the classification (e.g., "RED", "UNLIT").

Part 2: Autonomous Lane Detection (Hough Lines)

Input Directory: images/line_detection/

You are provided with a sequential set of images captured from a moving vehicle. You must implement the Polar Hough Line Transform to detect the primary ego-lanes (the left and right lane boundaries).

Objectives:

1. **Preprocessing:** Implement custom preprocessing to isolate lane markings (e.g., color thresholding, Gaussian blurring, and Region of Interest masking to ignore the sky and car hood).
2. **Edge Detection:** Apply your from-scratch edge detector to the preprocessed image.
3. **Hough Line Transform:** Implement the standard polar Hough Transform (ρ, θ) from scratch as detailed in the lectures.
4. **Lane Overlay:** Process the raw Hough accumulator peaks to extract finite line segments. Average these segments to form a single continuous left lane and a single continuous right lane. Fill the drivable area between them with a semi-transparent colored polygon.

Requirements & Deliverables:

- **Animated Output (GIF Requirement):** You must process **all** images in the directory sequentially. Combine the final overlaid outputs into a single animated .gif file running at 2 frame per second (2 fps).

Part 3: Real Object Localization (Generalized Hough Transform)

Input Directory: images/object_localization/

In this section, you will implement the Generalized Hough Transform (GHT) to localize a real object with an arbitrary, non-parametric shape. The goal is to detect a specific flower using only its edge geometry.

Objectives:

1. **Edge & Gradient Detection:** Implement an edge detection method from scratch for both the template image and the test image. You must compute both the edge magnitude and the gradient direction (ϕ).
2. **Reference Point Selection:** Select a reference center point (x_c, y_c) for the template object (e.g., the spatial center of the template image or the centroid of the edge pixels).
3. **ϕ-Table (R-Table) Construction:** Build the lookup table from the template image as taught in the lectures. For each edge pixel in the template, use its gradient direction ϕ_i to store the displacement vector $\vec{r} = (r, \alpha)$ to the reference point.
4. **Generalized Hough Voting:** Apply the voting process to the test image. For each edge pixel in the test image, use its gradient orientation to retrieve the corresponding displacement vectors from your ϕ-Table and cast votes in the 2D accumulator space $A(x_c, y_c)$.
5. **Object Localization:** Detect the strongest peak(s) in the accumulator to find the estimated location of the target object in the test image.

Assumptions & Constraints:

- The target object appears with an approximately fixed scale and rotation. Handling scale changes (S) or rotation (θ) in the accumulator is **not** required.
- You only need to detect the single strongest instance of the target object.
- **Forbidden:** You may not use built-in template matching (e.g., SSD, cross-correlation) or feature matching (SIFT/SURF). Localization *must* be done via the Generalized Hough voting accumulator.

Deliverable Visuals for Part 3:

You must include the following visual progression in your report:
Original Test Image → Grayscale Test Image → Edge Map → Hough Accumulator Heatmap → Final Detected Location *(Shown as a point, bounding box, or marker overlaid on the original color image test_flower_original.jpg)*.

Deadline: 5 June 2026

Report Structure and Deliverables

Submit a single .zip file containing your source code and an academic PDF report. Naming convention: HW3_Student1_Student2_Student3.zip.

Your PDF Report Must Include:

1. **Methodology:**
    - Mathematical explanation of your Hough Transform implementation.
    - Explanation of the vectorization techniques used to eliminate slow for loops.
2. **Performance Analysis:**
    - Provide a table detailing the execution time (in seconds) for each stage of your algorithm (Edge Detection, Hough Voting, Post-processing) for an average image in both parts.
3. **Results & Visualizations:**
    - Include step-by-step visual outputs (Original → Edge Map → Hough Accumulator Heatmap → Final Overlay).
4. **Conclusion:**
    - Summary of the challenges faced in strictly from-scratch implementations.